

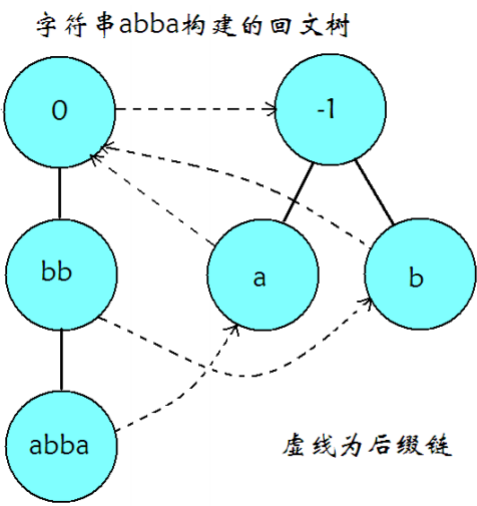
回文树

定义

回文树（EER Tree, Palindromic Tree, 也被称为回文自动机）是一种可以存储一个串中所有回文子串的高效数据结构。最初由 Mikhail Rubinchik 和 Arseny M. Shur 在 2015 年发表。它的灵感来源于后缀树等字符串后缀数据结构，使用回文树可以简单高效地解决一系列涉及回文串的问题。

结构

回文树大概长这样



和其它自动机类似的，回文树也是由转移边和后缀链接（fail 指针）组成，每个节点都可以代表一个回文子串。

因为回文串长度分为奇数和偶数，我们可以像 manacher 那样加入一个不在字符集中的字符（如 '#'）作为分隔符来将所有回文串的长度都变为奇数，但是这样过于麻烦了。有没有更好的办法呢？

答案自然是有。更好的办法就是建两棵树，一棵树中的节点对应的回文子串长度均为奇数，另一棵树中的节点对应的回文子串长度均为偶数。

和其它的自动机一样，一个节点的 fail 指针指向的是这个节点所代表的回文串的最长回文后缀所对应的节点，但是转移边并非代表在原节点代表的回文串后加一个字符，而是表示在原节点代表的回文串前后各加一个相同的字符（不难理解，因为要保证存的是回文串）。

我们还需要在每个节点上维护此节点对应回文子串的长度 len，这个信息保证了我们可以轻松地构造出回文树。

建造

回文树有两个初始状态，分别代表长度为 0 的回文串。我们可以称它们为奇根，偶根。它们不表示任何实际的字符串，仅作为初始状态存在，这与其他自动机的根节点是异曲同工的。

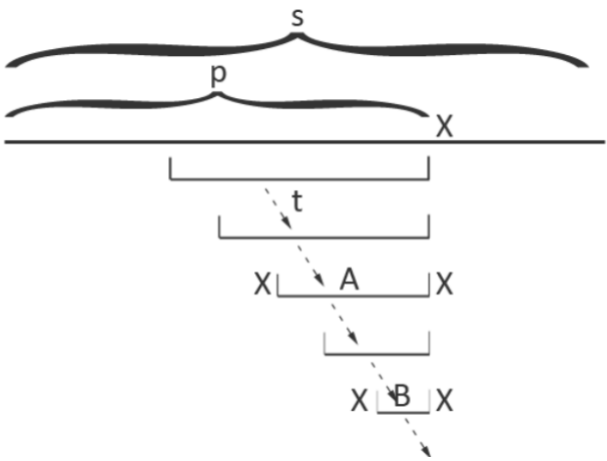
偶根的 fail 指针指向奇根，而我们并不关心奇根的 fail 指针，因为奇根不可能失配（奇根转移出的下一个状态长度为 1，即单个字符。一定是回文子串）

类似后缀自动机，我们增量构造回文树。

考虑构造完前 n 个字符的回文树后，向自动机中添加在原串里位置为 n 的字符。

我们从以上一个字符结尾的最长回文子串对应的节点开始，不断沿着 fail 指针走，直到找到一个节点满足，即满足此节点所对应回文子串的上一个字符与待添加字符相同。

这里贴出论文中的那张图



我们通过跳 fail 指针找到 A 所对应的节点，然后两边添加 x 就到了现在的回文串了（即 xAx），很显然，这个节点就是以 x 结尾的最长回文子串对应的树上节点。（同时，这个时候长度 1 节点优势出来了，如果没有 x 能匹配条件就是同一个位置的，就自然得到了代表字符 x 的节点。）此时要判断一下：没有这个节点，就需要新建。

然后我们还要求出新建的节点的 fail 指针。具体方法与上面的过程类似，不断跳转 fail 指针，从 A 出发，即可找到 xAx 的最长回文后缀 xBx，将对应节点设为 fail 指针所指的对象即可。

显然，这个节点是不需新建的， a 的前 位和后 位相同，都是 b ，前 位的两端根据回文串对应关系，都是 x ，后面被钦定了是 x ，于是这个节点 xbx 肯定已经被包含了。

如果 fail 没匹配到，那么将它连向长度为 的那个节点，显然这是可行的（因为这是所有节点的后缀）。

线性状态数证明

定理

对于一个字符串，它的本质不同回文子串个数最多只有 个。

证明

考虑使用数学归纳法。

- 当 时，只有一个字符，同时也只有一个子串，并且这个子串是回文的，因此结论成立。
- 当 时，设，其中 表示 最后增加一个字符 后形成的字符串，假设结论对 串成立。考虑以最后一个字符 结尾的回文子串，假设它们的左端点由小到大排序为 。由于 是回文串，因此对于所有位置，有 。所以，对于，已经在 中出现过。因此，每次增加一个字符，本质不同的回文子串个数最多增加 个。

由数学归纳法，可知该定理成立。

因此回文树状态数是 的。对于每一个状态，它实际只代表一个本质不同的回文子串，即转移到该节点的状态唯一，因此总转移数也是 的。

正确性证明

以上图为例，增加当前字符 x ，由线性状态数的证明，我们只需要找到包含最后一个字符 x 的最长回文后缀，也就是 xax 。继续寻找 xax 的最长回文后缀 xbx ，建立后缀链接。 xbx 对应状态已经在回文树中出现，包含最后一个字符的回文后缀就是 xax ， xbx 本身及其对应状态在 fail 树上的所有祖先。

对于 回文树的构造，令，显然除了跳 fail 指针的其他操作都是 的。

加入字符时，在上一次的基础上，每次跳 fail 后对应节点在 fail 树的深度，而连接 fail 后，仅为深度 + 1（但 fail 为 时（即到 才符合），深度相当于在 的基础上）。

因为只加入 个字符，所以只会加 次深度，最多也只会跳 次 fail。

因此，构造 的回文树的时间复杂度是 。

应用

本质不同回文子串个数

由线性状态数的证明，容易知道一个串的本质不同回文子串个数等于回文树的状态数（排除奇根和偶根两个状态）。

回文子串出现次数

建出回文树，使用类似后缀自动机统计出现次数的方法。

由于回文树的构造过程中，节点本身就是按照拓扑序插入，因此只需要逆序枚举所有状态，将当前状态的出现次数加到其 fail 指针对应状态的出现次数上即可。

例题：[「APIO2014」回文串](#)

定义 的一个子串的存在值为这个子串在 中出现的次数乘以这个子串的长度。对于给定的字符串，求所有回文子串中的最大存在值。

► 参考代码

最小回文划分

给定一个字符串，求最小的，使得存在，满足 均为回文串，且 依次连接后得到的字符串等于 。

考虑动态规划，记 表示 长度为 的前缀的最小划分数，转移只需要枚举以第 个字符结尾的所有回文串

为回文串

由于一个字符串最多会有 个回文子串，因此上述算法的时间复杂度为，无法接受，为了优化转移过程，下面给出一些引理。

记字符串 长度为 的前缀为，长度为 的后缀为 。

周期：若，，就称 是 的周期。

border：若，，就称 是 的 border。

周期和 border 的关系：是 的 border，当且仅当 是 的周期。

▼ 证明

若 是 的 border，那么，因此，所以 就是 的周期。

若 为 周期，则，因此，所以 是 的 border。

引理一

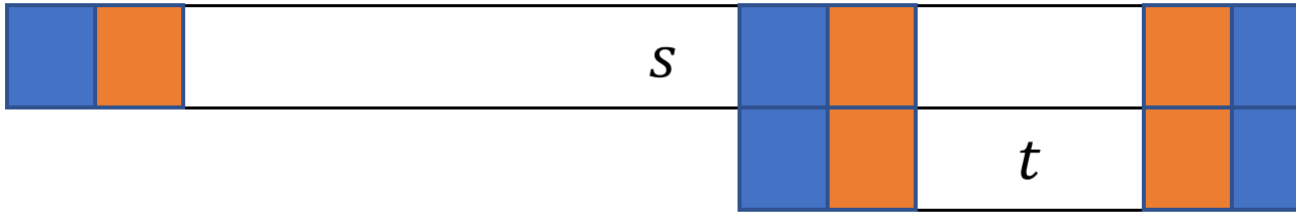
是回文串 的后缀，是 的 border 当且仅当 是回文串。

▼ 证明

对于，由 和 为回文串，因此有，所以 是 的 border。

对于，由 是 的 border，有，由 是回文串，有，因此，所以 是回文串。

下图中，相同颜色的位置表示字符对应相同。



引理二

是串 S 的 $\text{border}()$ ，是回文串当且仅当 S 是回文串。

▼ 证明

若 S 是回文串，由引理一， S 也是回文串。

若 S 是回文串，由 S 是 S 的 border ，因此，因为 S 是回文串，所以 S 也是回文串。

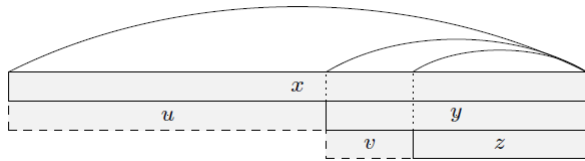
引理三

是回文串 S 的 border ，则 S 是 S 的周期，为 S 的最小周期，当且仅当 S 是最长回文真后缀。

引理四

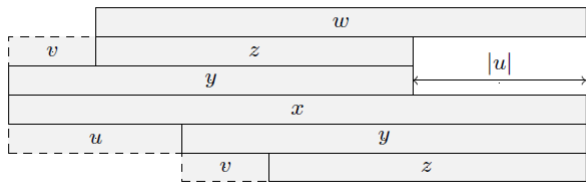
是一个回文串， S 是 S 的最长回文真后缀， T 是 T 的最长回文真后缀。令 $S = uv$ 分别为满足 S 的字符串，则有以下三条性质

1. u 是回文串；
2. 如果 v 是回文串，那么 S 是回文串；
3. 如果 S 是回文串，那么 u 是回文串。



▼ 证明

1. 由引理一， u 是 S 的最小周期， v 是 S 的最小周期。考虑反证法，假设 v 不是 S 的周期，因为 v 是 S 的后缀，所以 v 既是 S 的周期，也是 S 的周期，而 u 是 S 的最小周期，矛盾。所以 u 是 S 的周期。
2. 因为 S 是 S 的 border ，所以 S 是 S 的前缀，设字符串 $S = uv$ ，满足 $S = uv$ （如下图所示），其中 u 是 S 的 border 。考虑反证法，假设 u 不是 S 的 border ，那么 u 不是 S 的 border ，所以由引理一， S 是回文串，由引理一， S 是 S 的 border ，又因为 u 不是 S 的 border ，所以 u 不是 S 的 border ，矛盾。所以 u 是 S 的 border 。
3. 都是 S 的前缀，所以 u 是 S 的 border 。



推论

的所有回文后缀按照长度排序后，可以划分成 n 段等差数列。

▼ 证明

设 S 的所有回文后缀长度从小到大排序为 $p_1, p_2, \dots, p_n$ 。对于任意 i ，若 p_i 是 S 的周期，则 p_i 构成一个等差数列。否则，由引理一，有 p_i 不是 S 的周期，且 p_i 不是 S 的周期。因此，若相邻两对回文后缀的长度之差发生变化，那么这个最大长度一定会相对于最小长度翻一倍。显然，长度翻倍最多只会发生 $\log n$ 次，也就是 S 的回文后缀长度可以划分成 $\log n$ 段等差数列。

该推论也可以通过使用弱周期引理，对 S 的最长回文后缀的所有 border 按照长度分类，考虑这 $\log n$ 组内每组的最长 border 进行证明。详细证明可以参考金策的《字符串算法选讲》和陈孙立的 2019 年 IOI 国家候选队论文《子串周期查询问题的相关算法及其应用》。

有了这个结论后，我们现在可以考虑如何优化 S 的转移。

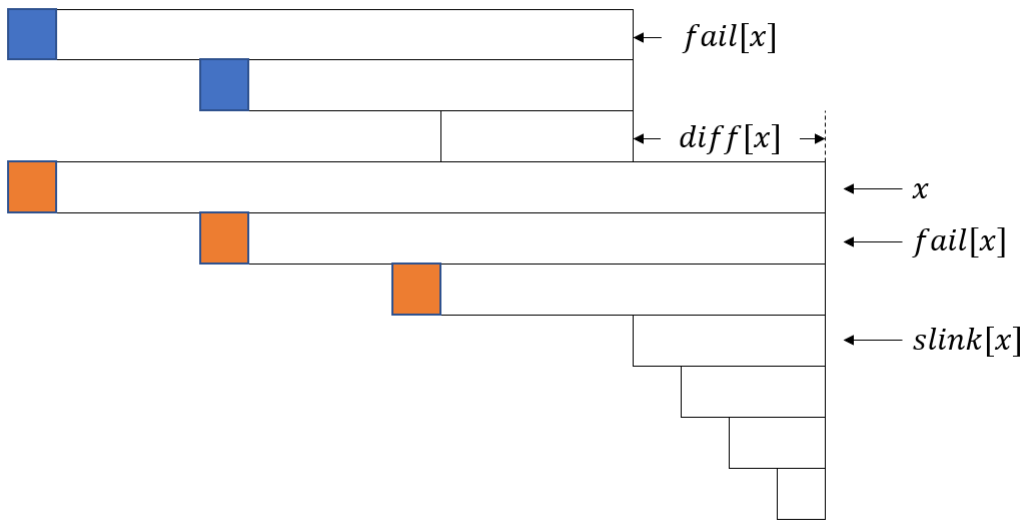
优化

回文树上的每个节点 u 需要多维护两个信息， fail 和 val 。表示节点 u 和 fail 所代表的回文串的长度差，即 $\text{val} = \text{len}(u) - \text{len}(\text{fail})$ 。表示 u 一直沿着 fail 向上跳到第一个节点，使得 val 是 fail 所在等差数列中长度最小的那个节点。

根据上面证明的结论，如果使用 fail 指针向上跳的话，每向后添加一个字符，只需要向上跳 $\log n$ 次。因此，可以考虑将一个等差数列表示的所有回文串的 val 之和（在原问题中指），记录到最长的那个回文串对应节点上。

表示 u 所在等差数列的 val 之和，且 u 是这个等差数列中长度最长的节点，则 val 是 u 当前枚举到的下标。

下面我们考虑如何更新 数组和 数组。以下图为例，假设当前枚举到第 个字符，回文树上对应节点为 。为橙色三个位置的 值之和（最短的回文串 算在下一个等差数列中）。上一次出现位置是 （在 处结束），包含的 值是蓝色位置。因此， 实际上等于 和多出来一个位置的 值之和，多出来的位置是。最后再用 去更新， 这部分等差数列的贡献就计算完毕了，不断跳， 重复这个过程即可。具体实现方式可参考例题代码。



最后，上述做法的正确性依赖于：如果 和 属于同一个等差数列，那么 上一次出现位置是 。

▼ 证明

根据引理， 是 的 border，因此其在 处出现。

假设 在 中的 位置出现。由于 和 属于同一个等差数列，因此。多余的 和 处的 有交集，记交集为，设串 满足。用类似引理 的方式可以证明， 是回文串，而 的前缀 也是回文串，这与 是最长回文前缀（后缀）矛盾。

例题：[Codeforces 932G Palindrome Partition](#)

给定一个字符串，要求将 划分为，其中 是偶数，且，求这样的划分方案数。

- 题解
- 参考代码

例题

- [最长双回文串](#)
- [拉拉队排练](#)
- [「SHOI2011」双倍回文](#)
- [HDU 5421 Victor and String](#)
- [CodeChef Palindromeness](#)

相关资料

- [EERTREE: An Efficient Data Structure for Processing Palindromes in Strings](#)
- [Palindromic tree](#)
- 2017 年 IOI 国家候选队论文集 回文树及其应用 翁文涛
- 2019 年 IOI 国家候选队论文集 子串周期查询问题的相关算法及其应用 陈孙立
- 字符串算法选讲 金策
- [A bit more about palindromes](#)
- [A Subquadratic Algorithm for Minimum Palindromic Factorization](#)

本页面最近更新：2024/3/27 06:46:12，[更新历史](#)

发现错误？想一起完善？[在 GitHub 上编辑此页！](#)

本页面贡献者：[aofall](#), [CoelacanthusHex](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [H-J-Granger](#), [Henry-ZHR](#), [huhao](#), [iamtwz](#), [Ir1d](#), [kenlig](#), [Leasier](#), [Marcythm](#), [NachtgeistW](#), [ouuan](#), [Persdre](#), [PotassiumWings](#), [shuzhouliu](#), [SukkaW](#), [Tiphereth-A](#), [TOMWT-qwq](#), [yjl9903](#), [ZXyaang](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用